

Un cliente de API para el resumidor

PF1822 · Especialización en Desarrollo con IA

Módulo 2 · MA04576 · INTRODUCCIÓN A MODELOS GENERATIVOS Y CONSUMO POR API · 21 h

TÉCNICA

Resolución de problemas

ORGANIZACIÓN

Individual; la parte A se discute en el aprendizaje esperado 1 y la parte B se revisa en el 2

INDICADORES DE LOGRO

1.2, 1.3, 2.1, 2.2, 2.3, 3.1, 3.2

APOYOS

Notebook guiado "Laboratorio de prompts"; Cuatro cápsulas de contenido, una por AE

PRODUCTODiagrama + `cliente_ia.py` + `test_cliente_ia.py` + `prompts.md`**SE EVALÚA CON**

Rúbrica de revisión de la solución (observación) (M2-Instrumento-1.pdf)

APRENDIZAJES ESPERADOS

AE1. ANALIZAR EL ROL DE LOS MODELOS GENERATIVOS EN EL DESARROLLO DE APLICACIONES, DESCRIBIENDO SUS TIPOS, CASOS DE USO, TECNOLOGÍAS ASOCIADAS Y REPRESENTANDO ARQUITECTURAS FUNCIONALES QUE INTEGREN ESTOS MODELOS MEDIANTE APIS.

AE2. UTILIZAR UN MODELO DE IA A TRAVÉS DE API REST UTILIZANDO SOLICITUDES HTTP Y AUTENTICACIÓN, BASÁNDOSE EN LA DOCUMENTACIÓN OFICIAL DE OPENAI Y HUGGING FACE.

AE3. FORMULAR PROMPTS EFECTIVOS PARA MODELOS GENERATIVOS EN ESCENARIOS ZERO-SHOT O FEW-SHOT, CONSIDERANDO LA TAREA SOLICITADA Y LOS PARÁMETROS DE GENERACIÓN DISPONIBLES.

Parte A — diagrama de referencia:**DIAGRAMA (MERMAID)**

```
flowchart LR
    U[App de la mesa de ayuda<br/>navegador] -->|ticket| S[Servidor API interno<br/>guarda la clave]
    S --> P[Preprocesamiento<br/>limpia HTML, firmas, datos personales]
    P --> M[Modelo de IA externo<br/>vía API REST]
    M -->|resumen| S
    S --> DB[(Base de datos /<br/>vector store)]
    S -->|resumen + prioridad| U
```

Componente**Función**

App de la mesa de ayuda

Muestra el ticket y su resumen; nunca conoce la clave.

Servidor API interno

Recibe el ticket, orquesta el flujo y es el único que guarda la clave (variable de entorno).

Preprocesamiento

Limpia el texto y quita datos personales antes de enviarlo: menos tokens y menos riesgo.

Modelo de IA externo

Genera el resumen y los embeddings.

Componente	Función
Base de datos / vector store	Guarda tickets, resúmenes y embeddings para buscar tickets parecidos.

Por qué la clave no va en el cliente: todo lo que llega al navegador puede leerlo cualquiera; una clave expuesta se usa para gastar a cuenta de la empresa. *Justificación modelo:* **seguridad**, porque la clave y los datos personales quedan del lado del servidor; **escalabilidad**, porque el servidor puede encolar tickets y procesarlos en lote sin cambiar la aplicación de la mesa de ayuda.

Parte B — `cliente_ia.py` **de referencia:**

```

"""Cliente mínimo y reutilizable para APIs de modelos generativos compatibles con
OpenAI."""
from __future__ import annotations

import os

import httpx

class ErrorAPI(Exception):
    """Error devuelto por la API, con su código HTTP y el mensaje del servicio."""

    def __init__(self, estado: int, mensaje: str):
        super().__init__(f"HTTP {estado}: {mensaje}")
        self.estado = estado
        self.mensaje = mensaje

class ClienteIA:
    """Encapsula las llamadas de listado de modelos, generación y embeddings.

    Args:
        base_url: URL base de la API, por ejemplo "https://api.openai.com/v1".
        clave_api: clave de acceso. Si es None, se lee de la variable
        OPENAI_API_KEY.
        modelo_chat: modelo de generación. Si es None, se lee de MODELO_CHAT.
        modelo_embeddings: modelo de embeddings. Si es None, se lee de
        MODELO_EMBEDDINGS.
        timeout: segundos máximos de espera por solicitud.
        http: cliente httpx ya construido; las pruebas lo usan para simular la API.
    """

    def __init__(self, base_url: str = "https://api.openai.com/v1", clave_api: str |
None = None,
                 modelo_chat: str | None = None, modelo_embeddings: str | None =
None,
                 timeout: float = 30.0, http: httpx.Client | None = None):
        self.clave_api = clave_api or os.environ.get("OPENAI_API_KEY")
        if not self.clave_api:
            raise ValueError("Falta OPENAI_API_KEY: defínela en el entorno, nunca en
el código.")
        self.modelo_chat = modelo_chat or os.environ.get("MODELO_CHAT")
        self.modelo_embeddings = modelo_embeddings or
os.environ.get("MODELO_EMBEDDINGS")
        self.http = http or httpx.Client(base_url=base_url, timeout=timeout)
        self.http.headers["Authorization"] = f"Bearer {self.clave_api}"

    def _revisar(self, respuesta: httpx.Response) -> dict:
        """Devuelve el JSON de la respuesta o lanza ErrorAPI si el estado es 400 o
más."""
        if respuesta.status_code >= 400:
            raise ErrorAPI(respuesta.status_code, respuesta.text[:300])
        return respuesta.json()

    def listar_modelos(self) -> list[str]:

```

```

    """Devuelve los identificadores de los modelos disponibles (GET /models)."""
    datos = self._revisar(self.http.get("/models"))
    return [m["id"] for m in datos.get("data", [])]

    def generar(self, mensajes: list[dict], temperature: float = 0.2, max_tokens:
int = 300) -> dict:
    """Envía una conversación al endpoint de chat (POST /chat/completions).

    Args:
        mensajes: lista de {"role": "system" | "user" | "assistant", "content":
str}.
        temperature: aleatoriedad de la respuesta; baja para resúmenes
consistentes.
        max_tokens: máximo de tokens de la respuesta.

    Returns:
        {"texto": str, "tokens": dict} con el texto generado y el uso de tokens.
    """
    if not self.modelo_chat:
        raise ValueError("Falta el modelo: define MODELO_CHAT.")
    datos = self._revisar(self.http.post("/chat/completions", json={
        "model": self.modelo_chat, "messages": mensajes,
        "temperature": temperature, "max_tokens": max_tokens,
    }))
    return {"texto": datos["choices"][0]["message"]["content"], "tokens":
datos.get("usage", {})}

    def embeddings(self, textos: list[str]) -> list[list[float]]:
    """Devuelve un vector por texto, en el mismo orden de entrada (POST
/embeddings)."""
    if not self.modelo_embeddings:
        raise ValueError("Falta el modelo: define MODELO_EMBEDDINGS.")
    datos = self._revisar(self.http.post("/embeddings", json={
        "model": self.modelo_embeddings, "input": textos,
    }))
    return [d["embedding"] for d in sorted(datos["data"], key=lambda d:
d["index"])]

```

`test_cliente_ia.py` **de referencia** (sin conexión, con `httpx.MockTransport`):

```

import httpx
import pytest

from cliente_ia import ClienteIA, ErrorAPI

def cliente_simulado(cuerpo: dict, estado: int = 200, registro: list | None = None)
-> ClienteIA:
    """Crea un ClienteIA cuyas solicitudes responde una API falsa."""
    def responder(solicitud: httpx.Request) -> httpx.Response:
        if registro is not None:
            registro.append(solicitud)
        return httpx.Response(estado, json=cuerpo)

    http = httpx.Client(base_url="https://api.test/v1",
transport=httpx.MockTransport(responder))
    return ClienteIA(clave_api="clave-de-prueba", modelo_chat="modelo-x",
        modelo_embeddings="embeddings-x", http=http)

def test_generar_devuelve_texto_tokens_y_autentica():
    registro = []
    cliente = cliente_simulado({"choices": [{"message": {"content": "Resumen"}}],
        "usage": {"total_tokens": 42}}, registro=registro)
    respuesta = cliente.generar([{"role": "user", "content": "hola"}])
    assert respuesta == {"texto": "Resumen", "tokens": {"total_tokens": 42}}
    assert registro[0].method == "POST"
    assert registro[0].url.path == "/v1/chat/completions"
    assert registro[0].headers["Authorization"] == "Bearer clave-de-prueba"

def test_embeddings_respeto_el_orden_de_entrada():
    cliente = cliente_simulado({"data": [{"index": 1, "embedding": [0.3]},
        {"index": 0, "embedding": [0.1]}]})
    assert cliente.embeddings(["a", "b"]) == [[0.1], [0.3]]

def test_listar_modelos_usa_get():
    registro = []
    cliente = cliente_simulado({"data": [{"id": "m1"}, {"id": "m2"}]},
registro=registro)
    assert cliente.listar_modelos() == ["m1", "m2"]
    assert registro[0].method == "GET"

def test_error_http_se_informa_con_su_codigo():
    cliente = cliente_simulado({"error": {"message": "clave inválida"}}, estado=401)
    with pytest.raises(ErrorAPI) as error:
        cliente.generar([{"role": "user", "content": "hola"}])
    assert error.value.estado == 401

def test_sin_clave_no_arranca(monkeypatch):
    monkeypatch.delenv("OPENAI_API_KEY", raising=False)

```

```
with pytest.raises(ValueError):  
    ClienteIA()
```

Salida esperada: 5 passed.

Uso con Hugging Face. La misma clase sirve con cualquier API compatible con el formato de OpenAI cambiando `base_url`, la clave y el modelo. **PENDIENTE:** el tutor confirma en la documentación vigente de Hugging Face (Inference Providers) la URL base compatible y un modelo de ejemplo antes de publicar la actividad; las URL de ese servicio han cambiado.

Nota sobre parámetros. Algunos modelos recientes piden `max_completion_tokens` en vez de `max_tokens` o no aceptan `temperature`. El participante lo comprueba en la documentación del modelo que use: es parte del indicador 2.1.

Parte C — respuesta modelada.

Diagnóstico:

1. **No dice qué es un buen resumen** (largo, foco, persona gramatical). Efecto: la salida (b) es tan corta que no nombra la plataforma ni la condición en que falla.
2. **No prohíbe agregar información.** Efecto: la salida (a) inventa una solución ("reinstalarla") que no está en el ticket.
3. **temperature 1,0 y sin límite de tokens** para una tarea que debe ser estable. Efecto: dos ejecuciones dan resultados muy distintos y el largo no está acotado.

Prompt corregido (zero-shot):

```
[system] Eres analista de soporte de Nube Sur. Resume el ticket en UNA oración de  
máximo  
25 palabras, en tercera persona. Indica qué falla, en qué producto y en qué  
condición  
ocurre. No propongas soluciones ni agregues datos que no estén en el ticket.  
[user] Ticket: <ticket limpio>
```

Parámetros: `temperature` 0,2, porque resumir es extraer y se busca la misma respuesta cada vez; límite de 60 tokens, porque una oración de 25 palabras cabe con margen.

Salidas esperadas (pueden variar en la redacción): "La app móvil se cierra al abrir el historial de pedidos en Android 14; en iOS funciona correctamente." y "El usuario no puede descargar la factura de agosto desde el portal porque el botón queda cargando."

Nivel logrado (indicadores 3.1 y 3.2): identifica al menos dos fallas con su efecto; el prompt corregido tiene rol, tarea, formato y restricción; los parámetros se justifican.

Errores típicos: la clave escrita en el código o en el notebook (falla grave, criterio 2 de la rúbrica); pruebas que llaman a la API real y fallan sin internet; no revisar el código de estado y leer `choices` de una respuesta de error. En la parte C: corregir el prompt agregando la solución correcta en vez de prohibir las soluciones.